# Unit-3

## Computer Programming Fundamentals

**Natural language is the language spoken by people, while programming language is intended for machines**. Both languages contain important similarities, such as the differentiation they make between syntax and semantics, their purpose to communicate and the existence of a basic composition.

A programming language defines a set of instructions that are compiled together to perform a specific task by the CPU. The programming language mainly refers to high-level languages such as C, C++, Pascal, COBOL, etc.

Each programming language contains a unique set of keywords and syntax, which are used to create a set of instructions. These languages vary in level of abstraction they provide from the hardware. Some programming languages provide less or no abstraction while some provide higher abstraction. Based on levels of abstraction, they can be classified into two categories:

- Low-level language
- High-level language

## Low-level language

The low-level language is a programming language that provides no abstraction from hardware and it is represented in 0 or 1 forms which are machine instructions. The languages that come in this category are Machine level language and Assembly language.

## Machine-level language

The machine-level language is a language that consists of a set of instructions that are in the binary form 0 or 1. As we know that computers can understand only machine instructions, which are in binary digits, i.e., 0 and 1, so instructions given to the computer can be only in binary codes. Creating a program in a machine-level language is a very difficult task as it is not easy for the programmers to write the program in machine instructions. It is error-prone as it is not easy to understand, and its maintenance is also very high. A machine-level language is not portable as each computer has its machine instructions, so if we write a program in one computer will no longer be valid in another computer.

The different processor architectures use different machine codes, for example, a PowerPC processor contains RISC architecture, which requires different code than intel x86 processor, which has a CISC architecture.

## Assembly Language

The assembly language contains some human-readable commands such as mov, add, sub. The problems which we face in machine-level language are reduced to some extent by using extended form of machine-level language known as assembly language. Since assembly language instructions are written in English words like mov, add, sub, so it is easier to write and understand.

Assembly language code is not portable because data is stored in computer registers and computer has to know different sets of registers. Computers can only understand machine-level instructions so we require a translator that converts assembly code into machine code. The translator used for translating code is known as **assembler**. Machine language provides no abstraction, assembly language provides less abstraction whereas high-level language provides a higher level of abstraction.

| Machine-level language | Assembly language |
|---|---|
| The machine-level language comes at the lowest level in the hierarchy, so it has zero abstraction level from the hardware. | The assembly language comes above the machine language means that it has less abstraction level from the hardware. |
| It cannot be easily understood by humans. | It is easy to read, write, and maintain. |
| The machine-level language is written in binary digits, i.e., 0 and 1. | The assembly language is written in simple English language, so it is easily understandable by the users. |

| | |
|---|---|
| It does not require any translator as the machine code is directly executed by the computer. | In assembly language, the assembler is used to convert the assembly code into machine code. |
| It is a first-generation programming language. | It is a second-generation programming language. |

## High-Level Language

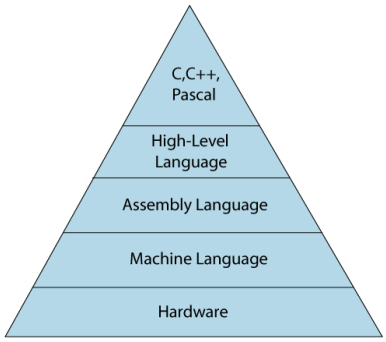
The high-level language is a programming language that allows a programmer to write the programs which are independent of a particular type of computer. The high-level languages are considered as high-level because they are closer to human languages than machine-level languages. When writing a program in a high-level language, then the whole attention needs to be paid to the logic of the problem. A **compiler** is required to translate a high-level language into a low-level language.

**Commonly used high-level languages**

- Python
- C++
- Visual Basic
- Java
- C#
- JavaScript

**Advantages of a high-level language**

- They are easy to read, write, maintain as it is written in English like words.
- They are designed to overcome limitation of low-level language i.e., portability.
- These languages are machine-independent.




| Low-level language | High-level language |
|---|---|
| It is a machine-friendly language, i.e., the computer understands the machine language, which is represented in 0 or 1. | It is a user-friendly language as this language is written in simple English words, which can be easily understood by humans. |
| The low-level language takes more time to execute. | It executes at a faster pace. |
| It requires the assembler to convert the assembly code into machine code. | It requires the compiler to convert the high-level language instructions into machine code. |
| The machine code cannot run on all machines, so it is not a portable language. | The high-level code can run all the platforms, so it is a portable language. |
| It is memory efficient. | It is less memory efficient. |
| Debugging and maintenance are not easier in a low-level language. | Debugging and maintenance are easier in a high-level language. |

## Fourth Generation Language

These are languages that consist of statements that are similar to statements in human language. These are used mainly in database programming and scripting. The fourth-generation language is also called a non procedural language. It enables users to access the database. Example Perl, Python, Ruby, SQL, MatLab.

**Advantages**

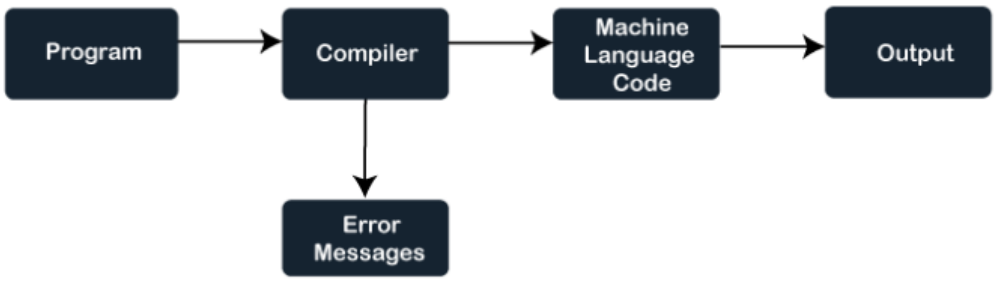
1. Easy to understand & learn.
2. Less time is required for application creation.
3. It is less prone to errors.

**Disadvantages**

1. Memory consumption is high.
2. Has poor control over Hardware.
3. Less flexible.

## Compiler

- A compiler is a translator that converts the high-level language into the machine language.
- Compiler is used to show errors to the programmer.
- The main purpose of compiler is to change the code written in one language without changing the meaning of the program.
- When you execute a program which is written in HLL programming language then it executes into two parts.



## Advantages of Compiler

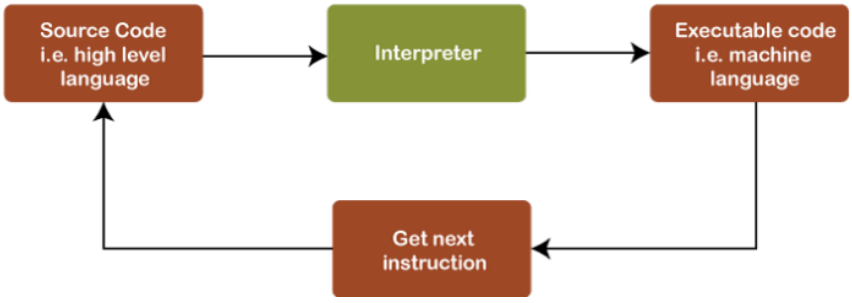
- A compiler translates a program in a single run.
- It consumes less time.
- CPU utilization is more.
- Both syntactic and semantic errors can be checked.
- It is easily supported by many high-level languages like C, C++, JAVA, etc.

## Interpreter

An interpreter is also a software program that translates a source code into a machine language. However, an interpreter converts high-level programming language into machine language **line-by-line** while interpreting and running the program.

## Advantages of Interpreter

- An interpreter translates the program line by line.
- The interpreter is smaller in size.
- Interpreter makes it easier to work with source code.
- So it is highly preferred, especially for beginners.
- Error localization is easier.



| Compiler | Interpreter |
|---|---|
| A compiler translates the entire source code in a single run. | An interpreter translates the entire source code line by line. |
| It consumes less time i.e., it is faster than an interpreter. | It consumes much more time than the compiler i.e., it is slower than the compiler. |
| It is more efficient. | It is less efficient. |
| CPU utilization is more. | CPU utilization is less as compared to the compiler. |
| Both syntactic and semantic errors can be checked simultaneously. | Only syntactic errors are checked. |
| The compiler is larger. | Interpreters are often smaller than compilers. |
| It is not flexible. | It is flexible. |
| The localization of errors is difficult. | The localization of error is easier than the compiler. |
| A presence of an error can cause the whole program to be re-organized. | A presence of an error causes only a part of the program to be re-organized. |
| The compiler is used by the language such | An interpreter is used by languages such as |
| C, C++ | Python, Javascript |

## Assembler

***Assembler*** is a computer system program that is utilized to convert assembly code into system or machine code that can be relocated. It is a type of low-level computer language. It sends instructions to the processors for various tasks and is unique to each processor. It is difficult to develop programs in it because the machine language only consists of ***0s*** and ***1s (Bit format)***. The assembly language is

similar to a machine language but has a simpler language and code.

As a result, users require an assembler to translate assembly code to machine code. It doesn't resolve all external references in the assembly code during the code translation. Later, the linker resolves these references. As a result, it is quite fast at translation. It creates machine language code in the form of an executable file. As a result, it required memory for the storing of this program.


**Advantages**

**1. Control** enable developers more control over their program's hardware and memory layout, which may be useful for optimizing performance and accessing low-level system features.

**2. Size**

It may be smaller in size than programs written in higher-level languages as they do not utilize libraries or other support codes.

**3. Speed** Assembly language programs may run faster than higher-level language programs as they are closer to the machine code that the computer

runs and don't need the extra translation steps that higher-level languages perform.

**4. System Portability** Assemblers may be more portable than those programs that are written in higher-level languages as they do not depend on specialized libraries or runtime environments.

**Disadvantages**

**1. Complexity** Assemblers are difficult to write and debug because they lack high-level language syntax to help catch problems.

**2. Vulnerability** Assembler programs are more vulnerable to faults and problems than high-level language programs because they need programmer to manage low-level features of computer's architecture manually.

**3. Speed** These are often slower and less efficient than high-level language programs that are compiled into machine code.

**4. Execution** These are machine-dependent, which means they may only execute on a certain type of computer system. It can make it complex to transfer programs from one system to another.

**5. Readability** These are frequently more complex to read and understand than programs that are written in high-level languages.

| PARAMETERS | COMPILER | INTERPRETER | ASSEMBLER |
|---|---|---|---|
| **Conversion** | It converts the high-defined programming language into Machine language or binary code. | It also converts the program-developed code into machine language or binary code. | It converts programs written in the assembly language to the machine language or binary code. |
| **Scanning** | It scans the entire program before converting it into binary code. | It translates the program line by line to the equivalent machine code. | It converts the source code into the object code then converts it into the machine code. |
| **Error Detection** | Gives the full error report after the whole scan. | Detects error line by line. And stops scanning until the error in the previous line is solved. | It detects errors in the first phase, after fixation the second phase starts. |
| **Code generation** | Intermediate code generation is done in the case of Compiler. | There is no intermediate code generation. | There is an intermediate object code generation. |
| **Execution time** | It takes less execution time comparing to an interpreter. | An interpreter takes more execution time than the compiler. | It takes more time than the compiler. |
| **Examples** | C, C#, Java, C++ | Python, Perl, VB, | GAS, GNU |

## Linker

A linker is special program that combines the object files, generated by compiler/assembler and other pieces of code to originate an executable file has .exe extension. In the object file, linker searches and append all libraries needed for execution of file. It regulates the memory space that will hold the code from each module. It also merges two or more separate object programs and establishes link among them.

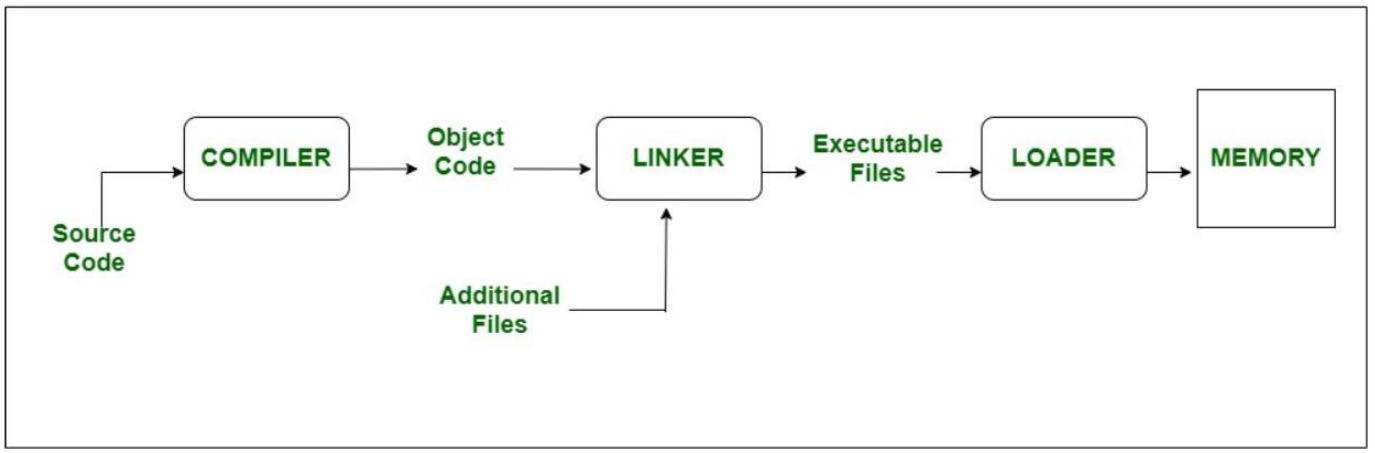
Linking is of two types :

1. Static Linking: Static linking is a kind of linking that is performed during the compilation of a source program in which linking is performed before the execution of the file or object. The linker produces a result at time of copying all library routines into executable image, which is known as static linking. Two major tasks performed by static linker are-
    - Symbol resolution: In this, each symbol has a predefined task and it associates each symbol exactly with one symbol definition from which they belong to.
    - Relocation: Its function is to modify symbol references to the relocated memory location and relocate the code and data section.
2. Dynamic linking: Dynamic linking is performed at the run time, in which multiple programs can share a single copy of the library. It means each module having the same object can share information of an object with other modules rather than linking the same object repeatedly into the library.

**The linker performs several tasks, including:**

- Symbol resolution: The linker resolves symbols in the program that are defined in one module and referenced in another.

- Code optimization: linker optimizes code generated by compiler to reduce code size and improve program performance.
- Memory management: The linker assigns memory addresses to the code and data sections of the program and resolves any conflicts that arise.
- Library management: linker can link external libraries into executable file to provide additional functionality.




## Loader

It is special program that takes input of executable files from linker, loads it to main memory, and prepares this code for execution by computer. Loader allocates memory space to program. It is in charge of loading programs and libraries in operating system. The embedded computer systems don't have loaders. In them, code is executed through ROM. There are following various loading schemes:

1. Absolute Loaders
2. Relocating Loaders
3. Direct Linking Loaders
4. Bootstrap Loaders

**The loader performs several tasks, including:**

- Loading: The loader loads the executable file into memory and allocates memory for the program.
- Relocation: The loader adjusts the program's memory addresses to reflect its location in memory.
- Symbol resolution: loader resolves any unresolved external symbols that are required by the program.
- Dynamic linking: loader can dynamically link libraries into program at runtime to provide additional functionality.

| LINKER | LOADER |
|---|---|
| The linker generates executable files of a source program. | The primary function of the loader is to load the executable module to the main memory. |
| An assembler generates the object code, which is taken as an input by the linker. | The linker creates the executable module, which is taken by the loader. |
| To generate an executable code, the linker combines all the object modules and source code. | In the main memory, it loads executable codes for further execution. |
| Linkage Editor and Dynamic linker are the two types of linker. | Absolute loading, Dynamic Run-time loading, and Relocatable loading are three kinds of loader. |
| Combining all object modules is another use of a linker. | The loader allocates the address to executable files. |
| Furthermore, in the program's address space, it is also responsible for arranging objects. | The references used within the program are handled by the loader. |

## Characteristics of Good programming Language

There are many programming languages, each corresponding to specific needs (formula calculus, character string processing, real-time, etc.) with each having specific characteristics and functionalities. Therefore, choice of programming language depends on the requirements to be fulfilled as well as the existing resources for understanding and training in the language.

1. **Simplicity:** A good programming language must be simple and easy to learn and use.
2. **Efficiency:** A program written in a programming language enables a computer system to translate it into machine code efficiently, execute it
3. **Portability**- A portable language is one which is implemented in variety of computers (machine dependent). Well defined language are more portable than others e.g. C, C++.
4. **Readability** − An essential principle for determining a programming language is the ease with which the programs can be read and learn.
5. **Extensibility**: A good programming language should allow extension through simple, natural, and elegant mechanisms.

6.**Documentation-** A programming language should be well structured and documented so that it is suitable for application development.
8.**Abstraction-** Abstraction is a must-have Characteristics for a programming language in which the ability to define the complex structure and hides unnecessary details.
9.**Consistent-** A programming language must be consistent in terms of syntax and semantics.

## Planning Computer Program

**Problem solving** is the process of defining a problem, identifying its root cause, prioritizing and selecting potential solutions, and implementing chosen solution.

**Defining the problem** means that you are diagnosing the situation. This helps take the further steps for solving the problem. Here you take effective measures to keep track of the situation of the problem. The Problem-Definition Process encourages you to define and understand the problem that you're trying to solve, in detail. The problem definition process helps to visualise the problem, by presenting it from different angles.

**Program design** consists of the steps a programmer should do before they start coding the program in a specific language. These steps when properly

documented will make the completed program easier for other programmers to maintain in the future. There are several methods or tools for planning the logic of a program. They include flowcharting, hierarchy or structure charts, pseudo code etc.

## Debugging

Debugging is the process of finding and fixing errors or bugs in the source code of any software. When software does not work as expected, computer programmers study the code to determine why any errors occurred.

### Steps involved in Debugging

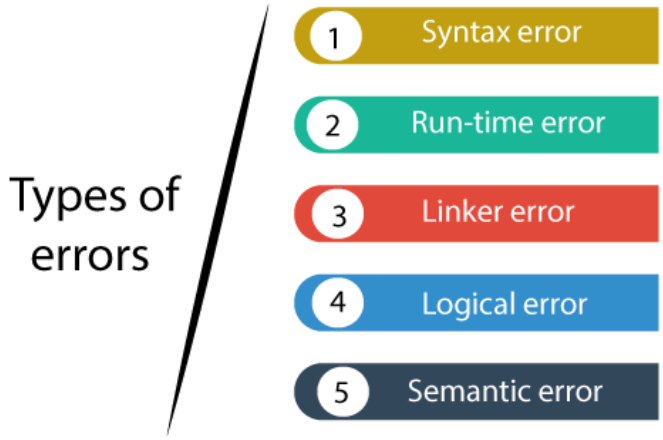
1. **Identify the Error:** It is obvious that the production errors reported by users are hard to interpret, and sometimes the information we receive is misleading. Thus it is mandatory to identify actual error.
2. **Find the Error Location:** Once error is correctly discovered, you will be required to thoroughly review the code repeatedly to locate position of the error. This step focuses on finding error.
3. **Analyze the Error:** The third step comprises error analysis, a bottom-up approach that starts from the location of error followed by analyzing code. Mainly error analysis has two significant goals, evaluation of

errors all over again to find existing bugs and finding uncertainty of incoming damage in a fix.

4. **Fix & Validate:** The last stage is the fix and validation that emphasizes fixing the bugs followed by running all the test scripts to check whether they pass.

## Types of Errors

Errors refers to issues that occur in a program that results in unwanted behavior of program. Programming errors are also known as the bugs or faults, and the process of removing these bugs is known as **debugging**. Some common Types of Errors in Programming are




### 1.Syntax Error

A syntax error is the most common type of error in programming. It occurs when the programmer writes code that is not in accordance with the syntax of the programming language. Syntax errors are detected by

the compiler, and the compiler reports an error message to the programmer.

**Common examples of syntax errors include**

- Forgetting to add a semicolon
- Spelling a keyword incorrectly
- Missing a closing brace.

**For example:**

1. If we want to declare the variable of type integer,
2. **int** a; // this is the correct form
3. Int a; // this is an incorrect form.

**2.Run-time Error**

Sometimes the errors exist during the execution-time even after the successful compilation known as run-time errors. A run-time error occurs during execution of a program. These errors occur when a program tries to perform an illegal operation. The program compiles without errors, but when it runs, it encounters an error that causes it to terminate abnormally. Run-time errors are often difficult to detect because they do not show up until the program is executed.

**Common examples of syntax errors include**

- dividing by zero
- attempting to access an invalid memory address.

Example - **int** b=2/0;

**3.Linker Error**

A linker error occurs when a program references a function or variable that is not defined in the program or the libraries it is linked against. Linker errors typically occur when a program is compiled and linked. The linker will report an error if it cannot find the necessary files or libraries to link the program.

Example- The most common linker error that occurs is that we use **Main()** instead of **main().**

**4.Logical Error**

A logical error occurs when the program compiles and runs without any syntax, run-time, or linker errors, but the output is incorrect. These errors occur when the program's logic or algorithms are incorrect. Logical errors are challenging to detect because the program does not generate any error messages.

Example **if** (a > b) {
cout << "a is less than b";
}
**else**
{
cout << "b is less than a";
}

**5.Semantic Error**

Semantic errors are errors that occur when the code written by the programmer makes no sense to the compiler, even though it is syntactically correct. They are different from syntax errors, which indicate errors in the structure of the program, as semantic errors are related to the meaning and implementation of the program.

**For example,**

- Using a string instead of an integer or accessing an array index that is out of bounds
- Uninitialized variables and type incompatibility are other common types of semantic errors.

**Documentation**

- Any written text, illustrations or video that describe a software or program to its users is called program or software document.
- At various stages of development multiple documents may be created for different users. In fact, software documentation is a critical process in the overall software development process.
- In modular programming documentation becomes even more important because different modules of the software are developed by different teams. If anyone other than the development team wants to

or needs to understand a module, good and detailed documentation will make the task easier.

**Some guidelines for creating the documents –**

- Document should be from point of view of reader
- Document should be unambiguous
- There should be no repetition
- Industry standards should be used
- Documents should always be updated

**Advantages of Documentation**

- Keeps track of all parts of a software or program
- Maintenance is easier
- Programmers other than the developer can understand all aspects of software
- Improves overall quality of the software
- Assists in user training
- Ensures knowledge de-centralization, cutting costs and effort if people leave the system abruptly

**Example Documents**

- User manual – It describes instructions and procedures for end users to use the different features of the software.
- Operational manual – It describes all the operations being carried out and their inter-dependencies.

- Design Document − It gives an overview of the software and describes design elements in detail. It contains details like data flow diagrams, entity relationship diagrams, etc.
- Requirements Document − It has list of all the requirements of the system as well as an analysis of the requirements. It can have real life scenarios, etc.
- Technical Documentation − It is a documentation of actual programming components like algorithms, flowcharts, program codes, functional modules, etc.
- Testing Document − It records test plan, test cases, validation plan, verification plan, test results, etc.
- List of Known Bugs − Every software has bugs or errors that cannot be removed because either they were discovered very late or are harmless or will take more effort and time than necessary to rectify. These bugs are listed with program documentation so that they may be removed at a later date. Also they help the users, implementers and maintenance people if the bug is activated.

## Structured programming

In structured programming design, programs are broken into different functions these functions are also known as **modules**, **subprogram**, **subroutines** and **procedures**.

Each function is design to do a specific task with its own data and logic. Information can be passed from one function to another function through parameters. Many high level languages supported structure programming.
Structured programming minimized the chances of the function affecting another. It supported to write clearer programs. Its organization helped to understand the programming logic easily. So that one can easily understand the logic behind the programs. It also helps the new comers of any industrial technology company to understand the programs created by their senior workers of the industry. It also made debugging easier.
**Functional abstraction** was introduced with structured programming. Abstraction simply means that how able one can or we can say that it means the ability to look at something without knowing about its inner details. In structured programming, it is important to know that a given function satisfies its requirement and performs a specific task, how that task is performed is not important.

**Advantages of structured programming**

1. It is user friendly and easy to understand.
2. Similar to English vocabulary of words and symbols.
3. It is easier to learn.
4. They require less time to write.

5. They are easier to maintain.
6. These are mainly problem oriented rather than machine based.
7. Program written in a higher level language can be translated into many machine languages and therefore can run on any computer for which there exists an appropriate translator.
8. It is independent of machine on which it is used i.e. programs developed in high level languages can be run on any computer.

**Disadvantages of structured programming**

1. A high level language has to be translated into the machine language by translator and thus a price in computer time is paid.
2. The object code generated by a translator might be inefficient compared to an equivalent assembly language program.
3. Data type are proceeds in many functions in a structured program. When changes occur in those data types, the corresponding change must be made to every location that acts on those data types within the program. This is really a very time consuming task if the program is very large.
4. In a structured program, each programmer is assigned to build a specific set of functions and data

types. Since different programmers handle separate functions that have mutually shared data type. Other programmers in the team must reflect the changes in data types done by the programmer in data type handled. Otherwise, it requires rewriting several functions.

**Bottom-up Approach**

The bottom-up approach is development strategy that involves breaking down a complex system into smaller, more manageable components, and then building those components up into a larger, more comprehensive program. At the heart of the bottom-up approach is the idea of modularization, which involves dividing a program into smaller, self-contained units that can be developed and tested independently of one another. Each module should have a well-defined interface that specifies how it interacts with other modules in the system, and it should be implemented in such a way that it can be reused in other programs or projects.

**The bottom-up approach involves several steps:**

- **Identify Individual Components:** The first step is to identify the individual functions and data structures that will be needed to implement the program. This

may involve analysing requirements, reviewing existing code, or brainstorming with the development team.

- **Design and Implement Components:** Once the individual components have been identified, programmers can begin designing and implementing them. Each component should be designed to be modular, reusable, and easy to maintain.
- **Test Components:** As each component is developed, it should be tested to ensure that it works correctly. This may involve writing test cases, running unit tests, or manually testing the component.
- **Incrementally Integrate Components:** Once a component has been tested and verified to work correctly, it can be integrated into the larger program. This integration should be done incrementally, testing the program at each step to ensure that it works correctly.
- **Repeat Steps 2-4:** The process of designing, implementing, testing, and integrating components should be repeated for each individual component, gradually building up the larger program.

Example:

Let's consider an example of developing a calculator application using the bottom-up approach in C++. In this approach, we will first identify individual components

such as addition, subtraction, multiplication, and division functions. These components will be designed and implemented separately and then integrated to form the complete calculator application.

**Advantages**

**Modular Design:** This approach allows programmers to design and implement individual components that are modular and reusable. This makes code more flexible and easier to maintain, as changes can be made to individual components without affecting the entire program.

**Faster Development:** This approach enables faster development times as individual components can be developed in parallel by different team members. Because each component is designed and tested separately, the changes to one component will not impact other components in the program.

**Early Testing:** The bottom-up approach facilitates early testing of individual components, allowing errors to be caught and corrected early in development cycle. This makes the testing process more efficient and reduces the cost of fixing errors later in the development cycle.

**Incremental Integration:** This enables incremental integration of individual components, making it easier to detect and correct errors as they arise. This approach reduces the risk of errors being missed until the entire

program is complete, making it more difficult and costly to correct them.

**Disadvantages:**

**Lack of Overview:** The bottom-up approach does not provide overview of entire system from the beginning. This may result in overlooking some high-level requirements, leading to incomplete or incorrect system functionality.

**Time-consuming:** The bottom-up approach requires more time in the initial stages of development to identify and develop individual components. This can lead to longer development times for smaller programs.

**Difficulty in Design:** The bottom-up approach can be challenging to design and implement if the individual components are complex and have a large number of dependencies. This can make the process of integrating components more difficult and time-consuming.

**Top-Down Approach**

A top-down approach is a programming methodology that involves starting with an overview of the problem, breaking it down into smaller sub-problems, and then gradually building the solution by implementing each sub-problem in a hierarchical manner. The top-down approach starts with a high-level view of the program

and then works downwards to implement the individual components.

The problem is broken down into smaller parts, and each part is solved separately, then combined to form the complete solution.

The top-down approach is often used in large projects because it helps to manage complexity by organizing the code into logical units that can be developed, tested, and integrated independently. In a top-down approach, the first step is to identify the problem and understand its requirements. This involves analysing the problem statement and determining the inputs and outputs required by the program. Once the problem is clearly defined, the next step is to break it down into smaller, more manageable sub-problems.

For example, let's say we are tasked with building a program to calculate the average temperature of a city. In a top-down approach, we would start by breaking down the problem into smaller parts, such as:

- Collecting temperature data from various sources (e.g., weather stations, online APIs, etc.)
- Storing the temperature data in a data structure (e.g., an array or a database)
- Calculating the average temperature using the collected data
- Displaying the average temperature to the user

Once we have broken down the problem into these smaller sub-problems, we can start implementing each one of them separately. This involves writing code for each sub-problem and testing it to ensure that it works as expected.

One of the key benefits of a top-down approach is that it promotes modularity and reusability. By breaking the problem down into smaller sub-problems, we can identify common patterns and solutions that can be reused in other parts of the program. This reduces the amount of redundant code and makes the program easier to maintain and extend.

**Advantages:**

**Modularity:** This approach promotes modularity by breaking down a problem into smaller sub-problems. This makes it easier to maintain, test and modify the code since each module can be developed and tested independently.

**Reusability:** This approach also promotes reusability since common patterns and solutions can be identified and reused in other parts of the program.

**Parallel Development:** Since different developers can work on different modules of program simultaneously, parallel development is possible, which can significantly reduce the development time and increase productivity.

**Clear understanding of the problem:** The top-down approach helps to develop a clear understanding of the problem and its requirements before starting the implementation, which helps to ensure that the program meets the user's needs.

**Disadvantages:**

**Difficulty in predicting design upfront:** The top-down approach can be challenging to predict the design of the program upfront since new requirements may arise during implementation that may require modifications to the original design.

**Integration issues:** As each sub-problem is developed independently, it can be challenging to integrate the different parts of program. This requires careful planning and testing to ensure that program works as expected.

**Oversimplification of the problem:** Breaking down a problem into smaller sub-problems can sometimes oversimplify the problem, leading to a less effective solution.

**Increased complexity:** As the number of modules in the program increases, the complexity of the program also increases. This can make it difficult to manage and maintain the code.

| S.No. | Top-Down Approach | Bottom-Up Approach |
|---|---|---|
| 1. | In this approach, the problem is broken down into smaller parts. | In this approach, the smaller problems are solved. |
| 2. | It is generally used by structured programming languages such as C, COBOL, FORTRAN, etc. | It is generally used with object oriented programming paradigm such as C++, Java, Python, etc. |
| 3. | It is generally used with documentation of module and debugging code. | It is generally used in testing modules. |
| 4. | It does not require communication between modules. | It requires relatively more communication between modules. |
| 5. | It contains redundant information. | It does not contain redundant information. |
| 6. | Decomposition approach is used here. | Composition approach is used here. |